Développement Web et API en Go

Damien Pereira

Durée : 18 heures

Objectifs

- Comprendre la philosophie et les concepts de Go
- Maîtriser la syntaxe et les fondamentaux du langage
- Développer des applications Go structurées
- Créer des applications web et APIs performantes
- Gérer les bases de données et l'authentification
- Déployer des applications Go en production

Plan

2. Installation et premiers pas
3. Les fondamentaux de Go
4. Exercices pratiques : Prendre en main Go
5. Le protocole HTTP et le développement web
6. Créer un serveur web avec Go
7. TP Final : Développer une API REST complète

Go : Un langage moderne

Go (ou Golang) est un langage de programmation :

- ✓ **Compilé** : Performances natives, binaires standalone
- ✓ **Typé statiquement** : Sécurité et optimisations
- ✓ **Concurrent** : Goroutines et channels natifs
- ✓ **Simple** : Syntaxe minimaliste et claire
- ✓ **Moderne** : Garbage collector, gestion mémoire automatique



L'histoire de Go

2007 : Naissance du projet

- Créé par Robert Griesemer, Rob Pike et Ken Thompson (Google)
- Frustration avec C++, Java et Python
- Objectif : Combiner performance et productivité

2009 : Annonce publique (10 novembre)

- Open source dès le début
- Version 1.0 en mars 2012

Aujourd'hui :

- Version stable actuelle : Go 1.27+
- Utilisé par Google, Uber, Docker, Kubernetes, Netflix...
- Millions de développeurs dans le monde



La philosophie de Go

Simplicité :

- Peu de mots-clés (~25 vs 84 en C++)
- Pas d'héritage de classes
- Une seule façon de faire les choses

Clarté :

- Code lisible > code concis
- Convention over configuration
- Erreurs explicites (pas d'exceptions)

Efficacité :

- Compilation ultra-rapide
- Binaires optimisés
- Concurrence native et performante

Ce qui rend Go différent

vs Python :

- 10-100x plus rapide
- Typage statique
- Moins de bibliothèques tierces

vs Java :

- Compilation en secondes (vs minutes)
- Pas de JVM nécessaire
- Syntaxe plus simple

vs C/C++ :

- Garbage collector automatique
- Sécurité mémoire
- Concurrence simplifiée

Les forces de Go

Performance :

- Compilation native (pas d'interpréteur)
- Binaires optimisés et rapides
- Faible consommation mémoire

Concurrence :

- Goroutines ultra-légères (milliers simultanées)
- Channels pour communication sécurisée
- Idéal pour services réseau et APIs

Déploiement :

- Un seul binaire statique
- Multi-plateforme (Linux, Windows, macOS) - Pas de dépendances runtime

Productivité :

- Syntaxe simple et claire
- Bibliothèque standard riche - Outils intégrés (formatting, testing, doc)

Cas d'usage de Go

Infrastructure et Cloud :

- Docker : Conteneurisation
- Kubernetes : Orchestration
- Terraform : Infrastructure as Code

Services Web :

- APIs REST haute performance
- Microservices
- Serveurs WebSocket

DevOps et Outils :

- CLI tools
- Monitoring (Prometheus, Grafana)
- Systèmes distribués

Réseau :

- Proxies et load balancers, VPN et tunneling, Services temps réel



Qui utilise Go ?

GAFAM :

- Google (Kubernetes, YouTube backend)
- Meta (services internes)
- Netflix (serveurs de streaming)

Tech Companies :

- Uber (services backend)
- Twitch (serveurs chat)
- Dropbox (infrastructure)

Startups :

- Thousands using Go for APIs
- Rapid development + scalability

Open Source :

- Docker, Kubernetes, Prometheus
- Hugo, Caddy, InfluxDB
- Terraform, Vault, Consul

S2 — Installation et premiers pas

90 min · Installer Go, configurer son environnement, initialiser un module, écrire son premier programme et comprendre les outils.

Installation et premiers pas

Installation de Go — windows

```
# Télécharger depuis go.dev/dl
# Installer le .msi
# Ou via Chocolatey
choco install golang
```

Installation de Go — macOS

```
# Via Homebrew
brew install go

# Ou télécharger depuis go.dev/dl
```

Installation de Go — linux

```
# Ubuntu/Debian
sudo apt update
sudo apt install golang-go

# Ou télécharger depuis go.dev/dl
wget https://go.dev/dl/go1.21.x.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.21.x.linux-amd64.tar.gz
```

Configuration de l'environnement (1/3)

Ajouter Go au PATH (Linux/macOS) :

```
# Dans ~/.bashrc ou ~/.zshrc
export PATH=$PATH:/usr/local/go/bin
export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin
```

Configuration de l'environnement — vérifier l'installation

```
# Version de Go
go version
# Output: go version go1.21.x linux/amd64

# Configuration
go env
# Affiche toutes les variables d'environnement
```

Configuration de l'environnement — tester

```
# Créer un dossier de travail
mkdir -p ~/go/src/hello
cd ~/go/src/hello
```

Créer un premier projet Go (1/2)

Go Modules (depuis Go 1.11) :

```
# Créer un dossier
mkdir hello-go
cd hello-go

# Initialiser un module
go mod init hello-go
# Crée un fichier go.mod

# Vérifier
cat go.mod
```

Créer un premier projet Go — go.mod

```
module hello-go

go 1.21
```

Premier programme : Hello World — créer `main.go`

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Premier programme : Hello World — exécuter

```
# Exécuter directement
go run main.go
# Output: Hello, World!

# Compiler en binaire
go build -o hello main.go
./hello
# Output: Hello, World!
```

Structure d'un programme Go

```
package main              // 1. Déclaration du package

import "fmt"              // 2. Imports

func main() {             // 3. Fonction main
    fmt.Println("Hello")  // 4. Code
}
```

Explications :

- `package main` : Package exécutable (entry point)
- `import "fmt"` : Importer des bibliothèques
- `func main()` : Point d'entrée du programme
- `fmt.Println` : Fonction de la bibliothèque standard

Note : Pas de `;` nécessaire en fin de ligne !

Les outils Go

```
# Exécuter du code
go run main.go

# Compiler
go build
go build -o myapp main.go

# Formater le code (obligatoire !)
go fmt ./...

# Télécharger les dépendances
go mod download
go mod tidy

# Lancer les tests
go test ./...

# Documenter
go doc fmt.Println
```

Aide et documentation

```
# Documentation d'une fonction
go doc fmt.Println

# Documentation d'un package
go doc fmt

# Serveur de documentation local
go doc -http=:6060
# Ouvrir http://localhost:6060

# Documentation en ligne
# https://pkg.go.dev/
```

Ressources :

- **Go Tour** : https://go.dev/tour/
- **Documentation** : https://go.dev/doc/
- **Go by Example** : https://gobyexample.com/

S3 — Variables, types & collections

90 min · Déclaration de variables, types primitifs, tableaux, slices et maps.

Variables et constantes — syntaxe explicite

```
var name string = "Go"
var age int = 15
var price float64 = 9.99

// Sans initialisation (valeur zéro)
var count int       // 0
var message string  // ""
var active bool     // false
```

Variables et constantes (2/3)

Déclaration courte (dans les fonctions) :

```
name := "Go"
age := 15
price := 9.99

// Multiple
x, y := 10, 20
name, age := "Alice", 25
```

Variables et constantes — constantes

```
const PI = 3.14159
const AppName = "MyAPI"
const MaxUsers = 100

// Constantes typées
const Port int = 8080
```

Types de données primitifs — entiers

```
var i int = 42           // Taille dépend de la plateforme (32 ou 64 bits)
var i8 int8 = 127        // -128 à 127
var i16 int16 = 32767    // -32768 à 32767
var i32 int32 = 2147483647
var i64 int64 = 9223372036854775807

var u uint = 42          // Unsigned int
var u8 uint8 = 255       // 0 à 255 (byte)
var u32 uint32 = 4294967295
```

Types de données primitifs — flottants

```
var f32 float32 = 3.14
var f64 float64 = 3.14159265359  // Plus précis

var c64 complex64 = 1 + 2i
var c128 complex128 = 1 + 2i
```

Types de données (suite) — booléens

```
var b bool = true
var isActive bool = false

// Opérations
result := true && false  // false
result = true || false   // true
result = !true          // false
```

Types de données (suite) — chaînes de caractères

```
var s string = "Hello"
var message string = "Hello, World!"

// Concaténation
greeting := "Hello" + " " + "World"

// Multi-lignes
var text string = `
    Ceci est une
    chaîne multi-lignes
    en Go
`

// Accès aux caractères
first := message[0]  // byte (uint8)
```

Tableaux (Arrays) — déclaration

```
// Déclaration avec taille
var arr [5]int  // [0, 0, 0, 0, 0]

// Avec initialisation
var numbers [5]int = [5]int{1, 2, 3, 4, 5}

// Syntaxe courte
primes := [5]int{2, 3, 5, 7, 11}

// Taille automatique
auto := [...]int{1, 2, 3, 4, 5}  // [5]int
```

Tableaux (Arrays) — opérations

```
arr[0] = 10       // Modifier
value := arr[0]   // Accéder
length := len(arr) // Longueur

// Itérer
for i := 0; i < len(arr); i++ {
    fmt.Println(arr[i])
}
```

Limitation : Taille fixe, peu flexible

Slices (Tranches) — déclaration

```
// Slice vide
var slice []int

// Avec make
numbers := make([]int, 5)      // len=5, cap=5, [0,0,0,0,0]
numbers := make([]int, 5, 10)  // len=5, cap=10

// Littéral
slice := []int{1, 2, 3, 4, 5}

// Depuis un array
arr := [5]int{1, 2, 3, 4, 5}
slice := arr[1:4]  // [2, 3, 4]
```

Slices (Tranches) — opérations

```
// Ajouter des éléments
slice = append(slice, 6)
slice = append(slice, 7, 8, 9)

// Longueur et capacité
len(slice)  // Nombre d'éléments
cap(slice)  // Capacité totale

// Copier
dest := make([]int, len(slice))
copy(dest, slice)
```

Slices : Manipulation — extraire des portions

```
numbers := []int{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

subset := numbers[2:5]   // [2, 3, 4]
subset := numbers[:5]    // [0, 1, 2, 3, 4]
subset := numbers[5:]    // [5, 6, 7, 8, 9]
subset := numbers[:]     // Tout le slice

// Exclure le dernier
without := numbers[:len(numbers)-1]
```

Slices : Manipulation (2/2)

Attention : Les slices partagent la mémoire !

```
original := []int{1, 2, 3, 4, 5}
subset := original[1:4]  // [2, 3, 4]

subset[0] = 99
// original = [1, 99, 3, 4, 5]
```

Maps (Dictionnaires)

Déclaration :

```
// Avec make
users := make(map[string]int)
users["alice"] = 25
users["bob"] = 30

// Littéral
ages := map[string]int{
    "alice": 25,
    "bob":   30,
    "carol": 35,
}

// Map de structs
employees := map[int]Employee{
    1: {Name: "Alice", Age: 25},
    2: {Name: "Bob", Age: 30},
}
```

Maps : Opérations

Opérations courantes :

```
// Ajouter/Modifier
ages["david"] = 40

// Récupérer
age := ages["alice"]  // 25

// Vérifier existence
age, exists := ages["alice"]
if exists {
    fmt.Println("Alice a", age, "ans")
}

// Supprimer
delete(ages, "bob")

// Longueur
count := len(ages)

// Itérer
```

Structures (Structs) — déclaration

```
type User struct {
    ID       int
    Name     string
    Email    string
    Age      int
    IsActive bool
}

// Struct anonyme
var person struct {
    Name string
    Age  int
}
```

Structures (Structs) — création

```
// Avec noms de champs
user1 := User{
    ID:    1,
    Name:  "Alice",
    Email: "alice@example.com",
    Age:   25,
}

// Sans noms (ordre important)
user2 := User{1, "Bob", "bob@example.com", 30, true}

// Valeurs par défaut
user3 := User{Name: "Carol"}  // Autres champs = valeurs zéro
```

Structs : Utilisation — accéder aux champs

```
user := User{ID: 1, Name: "Alice", Age: 25}

// Lecture
fmt.Println(user.Name)   // Alice
fmt.Println(user.Age)    // 25

// Modification
user.Age = 26
user.Email = "alice@newdomain.com"

// Pointeurs
ptr := &user
ptr.Age = 27
// Pas besoin de (*ptr).Age, Go le fait automatiquement
```

Structs : Utilisation — structs imbriquées

```
type Address struct {
    Street  string
    City    string
    Country string
}

type Employee struct {
    Name    string
    Address Address
}

emp := Employee{
    Name: "Alice",
    Address: Address{
        Street:  "123 Main St",
        City:    "Paris",
        Country: "France",
    },
}

fmt.Println(emp.Address.City)  // Paris
```

S4 — Structs & pointeurs

90 min · Déclarer des structures, structs imbriquées, et manipuler des pointeurs.

Pointeurs

Un pointeur stocke l'**adresse mémoire** d'une variable.

```
// Variable normale
x := 42

// Pointeur vers x
ptr := &x  // Type: *int

// Déréférencement
value := *ptr  // 42

// Modification via pointeur
*ptr = 100
fmt.Println(x)  // 100
```

Pointeurs avec structs

Passer `*User` au lieu de `User` évite la copie **et** permet de modifier l'original.

```
type User struct {
    Name string
    Age  int
}

func updateAge(u *User, newAge int) {
    u.Age = newAge  // Modifie l'original
}

user := User{Name: "Alice", Age: 25}
updateAge(&user, 26)
fmt.Println(user.Age)  // 26
```

Fonctions en Go

Déclaration avec `func`, type de retour à droite.

```
// Fonction simple
func greet(name string) string {
    return "Hello, " + name
}

// Appel
message := greet("Alice")  // "Hello, Alice"
```

Fonctions : paramètres multiples

Même type → raccourci `(a, b int)`. Types différents → tout détailler.

```
// Même type
func add(a, b int) int {
    return a + b
}

// Types différents
func createUser(name string, age int, active bool) User {
    return User{Name: name, Age: age, IsActive: active}
}
```

Fonctions : retours multiples

Pattern Go idiomatique : **(résultat, error)**.

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zéro")
    }
    return a / b, nil
}

// Utilisation
result, err := divide(10, 2)
if err ≠ nil {
    fmt.Println("Erreur:", err)
} else {
    fmt.Println("Résultat:", result)  // 5
}
```

Fonctions : retours nommés

Les variables de retour sont déclarées dans la signature ; `return` nu les renvoie.

```
func getCoordinates() (x, y int) {
    x = 10
    y = 20
    return  // Retourne x et y automatiquement
}

x, y := getCoordinates()
```

Fonctions variadic

Un paramètre `...type` capte 0..N arguments.

```
func sum(numbers ...int) int {
    total := 0
    for _, n := range numbers {
        total += n
    }
    return total
}

sum(1, 2, 3)           // 6
sum(1, 2, 3, 4, 5)     // 15
```

Fonctions anonymes

Une fonction peut être stockée dans une variable ou exécutée inline.

```
// Fonction dans une variable
multiply := func(a, b int) int {
    return a * b
}
result := multiply(3, 4)  // 12

// Fonction inline (IIFE)
result := func(x int) int {
    return x * x
}(5)  // 25
```

S5 — Fonctions & contrôle de flux

90 min · Fonctions, retours multiples, variadic, if/switch, boucle for et range.

Contrôle de flux : If/Else — if simple

```
age := 20

if age ⩾ 18 {
    fmt.Println("Majeur")
}
```

Contrôle de flux : If/Else — if avec initialisation

```
if age := getAge(); age ⩾ 18 {
    fmt.Println("Majeur")
} else {
    fmt.Println("Mineur")
}
// age n'existe plus ici
```

Contrôle de flux : If/Else — If/Else if

```
score := 85

if score ⩾ 90 {
    fmt.Println("A")
} else if score ⩾ 80 {
    fmt.Println("B")
} else if score ⩾ 70 {
    fmt.Println("C")
} else {
    fmt.Println("D")
}
```

Contrôle de flux : Switch — switch classique

```
day := "Tuesday"

switch day {
case "Monday":
    fmt.Println("Début de semaine")
case "Tuesday", "Wednesday", "Thursday":
    fmt.Println("Milieu de semaine")
case "Friday":
    fmt.Println("Vendredi!")
default:
    fmt.Println("Weekend!")
}
```

Contrôle de flux : Switch — switch sans expression

```
score := 85

switch {
case score ⩾ 90:
    fmt.Println("Excellent")
case score ⩾ 70:
    fmt.Println("Bien")
default:
    fmt.Println("Peut mieux faire")
}
```

Boucles en Go — for classique

```
for i := 0; i < 5; i++ {
    fmt.Println(i)  // 0, 1, 2, 3, 4
}
```

Boucles en Go — for comme while

```
i := 0
for i < 5 {
    fmt.Println(i)
    i++
}
```

Boucles en Go — boucle infinie

```
for {
    // Code
    if condition {
        break
    }
}
```

Boucles en Go — for avec range

```
// Slice
numbers := []int{1, 2, 3, 4, 5}
for index, value := range numbers {
    fmt.Printf("Index: %d, Value: %d\n", index, value)
}

// Map
ages := map[string]int{"Alice": 25, "Bob": 30}
for name, age := range ages {
    fmt.Printf("%s: %d ans\n", name, age)
}
```

Méthodes — méthode avec receiver

```
type User struct {
    Name string
    Age  int
}

// Méthode avec receiver par valeur
func (u User) Greet() string {
    return "Hello, I'm " + u.Name
}

// Utilisation
user := User{Name: "Alice", Age: 25}
fmt.Println(user.Greet())  // "Hello, I'm Alice"
```

Méthodes — méthode avec pointer receiver

```
// Modifie l'original
func (u *User) SetAge(age int) {
    u.Age = age
}

// Utilisation
user := User{Name: "Alice", Age: 25}
user.SetAge(26)
fmt.Println(user.Age)  // 26
```

Quand utiliser un pointeur :

- Pour modifier le receiver
- Pour éviter de copier de grandes structures
- Par cohérence dans un type

Interfaces — définition

```
type Greeter interface {
    Greet() string
}

type Speaker interface {
    Speak() string
    Listen() string
}
```

Interfaces — implémentation implicite

```
type Person struct {
    Name string
}

// Person implémente Greeter automatiquement
func (p Person) Greet() string {
    return "Hi, I'm " + p.Name
}

type Robot struct {
    ID string
}

func (r Robot) Greet() string {
    return "Beep boop, I'm " + r.ID
}
```

Interfaces : Utilisation — utiliser une interface

```
func greetAll(greeters []Greeter) {
    for _, g := range greeters {
        fmt.Println(g.Greet())
    }
}

// Usage
person := Person{Name: "Alice"}
robot := Robot{ID: "R2D2"}

greeters := []Greeter{person, robot}
greetAll(greeters)
// Output:
// Hi, I'm Alice
// Beep boop, I'm R2D2
```

Interfaces : Utilisation — interface vide

```
// interface{} accepte n'importe quel type
func printAnything(v interface{}) {
    fmt.Println(v)
}

printAnything(42)
printAnything("Hello")
printAnything(User{Name: "Alice"})
```

Gestion des erreurs — error est une interface

```
type error interface {
    Error() string
}
```

Gestion des erreurs — pattern classique

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zéro")
    }
    return a / b, nil
}

// Utilisation
result, err := divide(10, 0)
if err ≠ nil {
    fmt.Println("Erreur:", err)
    return
}
fmt.Println("Résultat:", result)
```

Erreurs personnalisées — struct d'erreur

```
type ValidationError struct {
    Field   string
    Message string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("%s: %s", e.Field, e.Message)
}

// Utilisation
func validateAge(age int) error {
    if age < 0 {
        return &ValidationError{
            Field:   "age",
            Message: "doit être positif",
        }
    }
    return nil
}
```

Erreurs personnalisées — wrapping d'erreurs

```
func openConfig(filename string) error {
    file, err := os.Open(filename)
    if err ≠ nil {
        return fmt.Errorf("impossible d'ouvrir config: %w", err)
    }
    defer file.Close()
    return nil
}
```